



CS240 Algorithm Design and Analysis

Lecture 10

NP and Computational Intractability

Qingying Hao

Spring 2026

2026.04.02



Algorithm Design Patterns and Anti-Patterns



- **Algorithm design patterns**

- Greedy
- Divide-and-Conquer
- Dynamic Programming
- Max-flow Min-cut

- **Reductions**

- **Algorithm design anti-patterns**

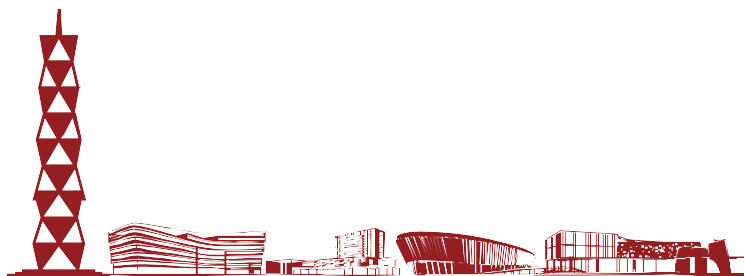
- **NP-completeness**

- PSPAC-completeness
- Undecidability

$O(n^k)$ algorithm unlikely

$O(n^k)$ certification algorithm unlikely

No algorithm possible





Polynomial-Time Reductions

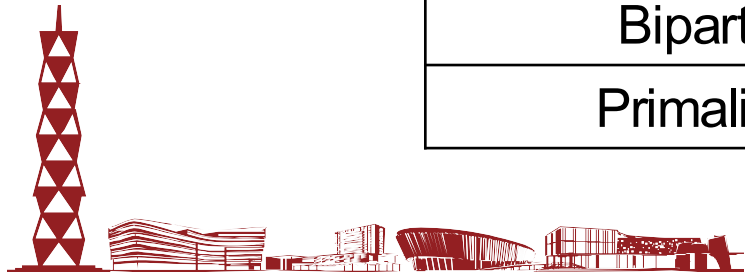




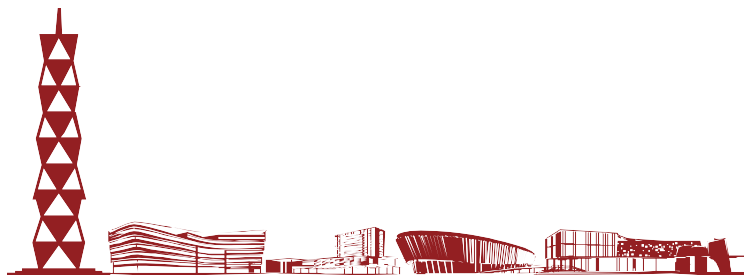
Classify Problems According to Computational Requirements

- **Q.** Which problems will we be able to solve in practice?
- **A working definition.** [Cobham 1964, Edmonds 1965, Rabin 1966]
- Those with polynomial-time algorithms. (In practice, poly-time algorithms scale to huge problems)

Yes	Probably no
Shortest path	Longest path
Matching	3D-matching
Min cut	Max cut
2-SAT	3-SAT
Planar 4-color	Planar 3-color
Bipartite vertex cover	Vertex cover
Primality testing (2002)	Factoring



- **Desiderata.** Classify problems according to those that can be solved in polynomial-time and those that cannot
- **Provably requires exponential-time**
 - Given a program of size $\log k$, does it halt in at most k steps?
 - Given a board position in an n -by- n generalization of checker, can black guarantee a win?
- **Frustrating news.** Huge number of fundamental problems have defied classification for decades
- Show that these fundamental problems are “computationally equivalent” and appear to be different manifestations of one **really hard** problem

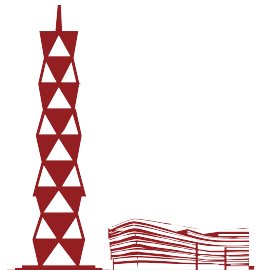
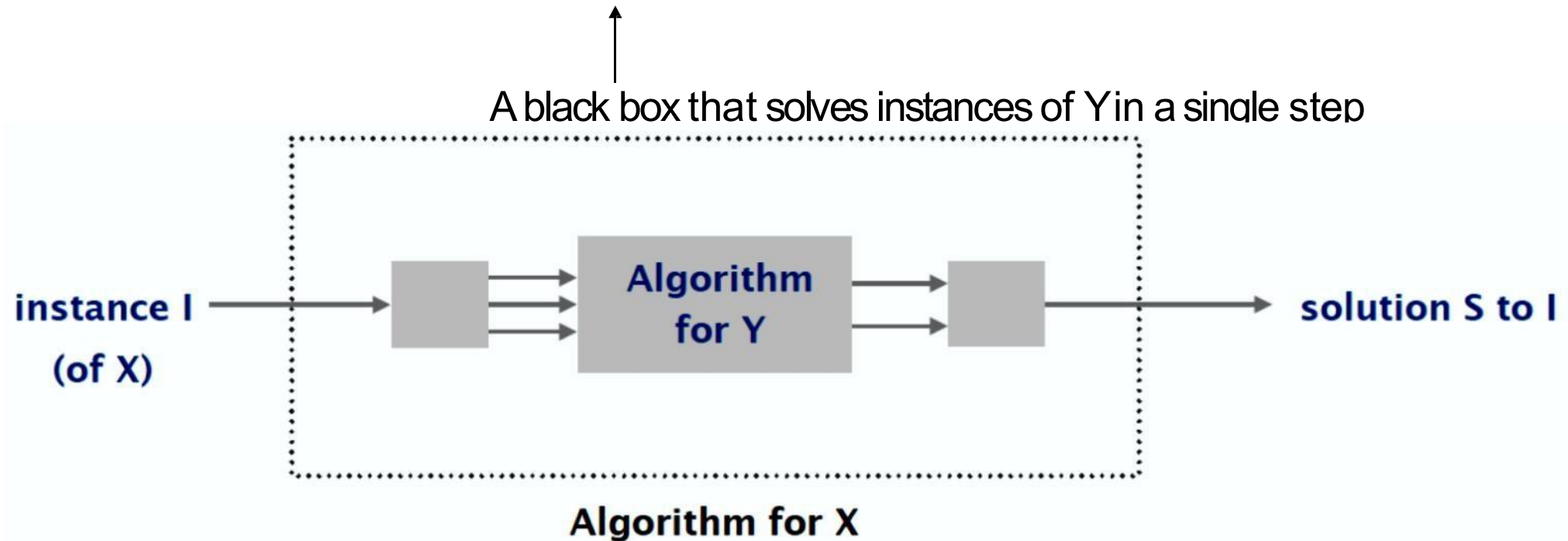




Polynomial-Time Reduction



- **Desiderata'**. Suppose we could solve X in polynomial-time. What else could we solve in polynomial time?
- **Reduction**. Problem X **polynomial-time reduces to** problem y if arbitrary instances of problem X can be solved using:
 - Polynomial number of standard computational steps, plus
 - Polynomial number of calls to oracle that solves problem Y





Polynomial-Time Reduction



- **Desiderata'**. Suppose we could solve X in polynomial-time. What else could we solve in polynomial time?

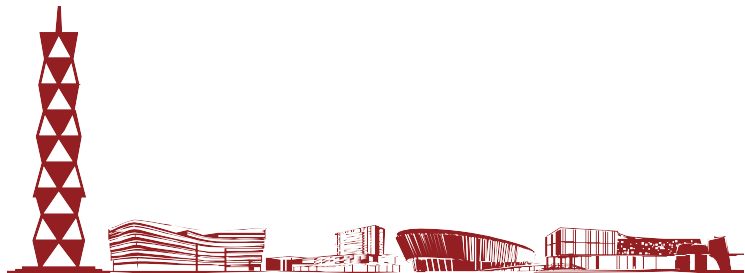
Don't confuse with reduces from

- Reduction. Problem X **polynomial-time reduces to** problem y if arbitrary instances of problem X can be solved using:
 - Polynomial number of standard computational steps, plus
 - Polynomial number of calls to oracle that solves problem Y



A black box that solves instances of Y in a single step

- **Notation.** $X \leq_p Y$
- **Remarks.** We pay for time to write down instances sent to black box $\rightarrow$ instances of y must be of polynomial size

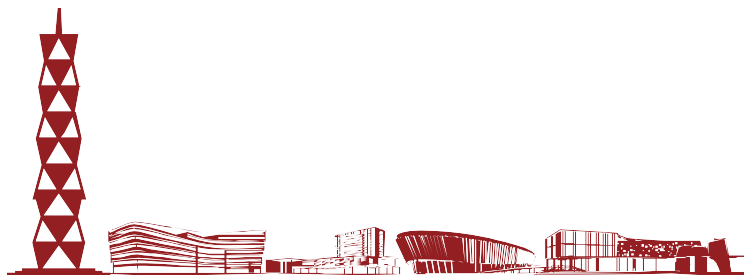




Polynomial-Time Reduction



- **Purpose.** Classify problems according to relative difficulty
- **Design algorithms.** If $X \leq_p y$ and y can be solved in polynomial-time, then X can also be solved in polynomial time
- **Establish intractability.** If $X \leq_p y$ and X cannot be solved in polynomial-time, then y cannot be solved in polynomial time
- **Establish equivalence.** If $X \leq_p y$ and $y \leq_p X$, we use notation $X \equiv_p y$. In this case, X can be solved in polynomial time iff y can be

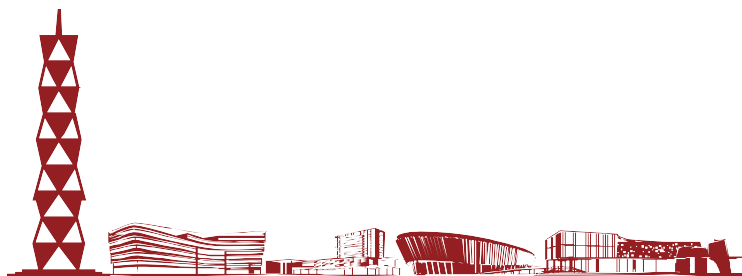




Reduction by Simple Equivalence



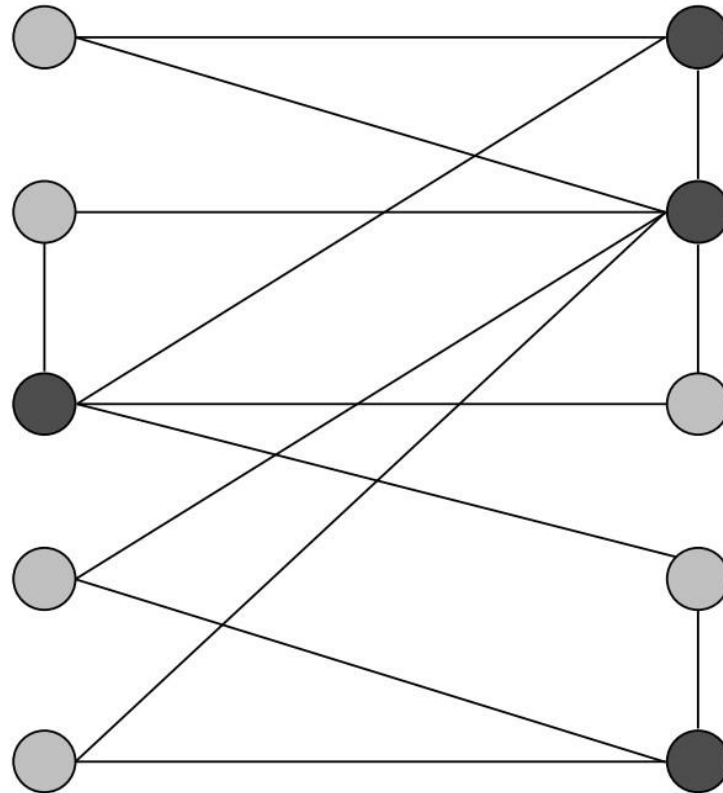
- **Basic reduction strategies**
 - **Reduction by simple equivalence**
 - Reduction from special case to general case
 - Reduction by encoding with gadgets



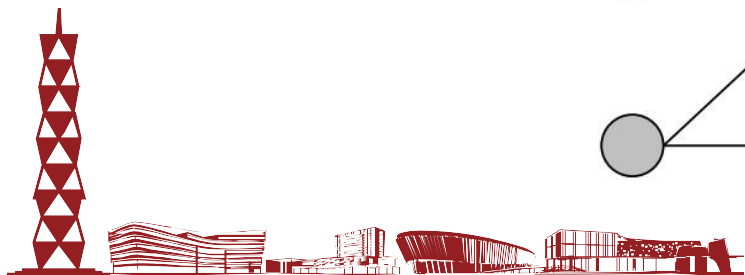
Independent Set



- **Independent Set:** Given a graph $G = (V, E)$ and an integer k , is there a subset of vertices $S \subseteq V$ such that $|S| \geq k$, and for each edge at most one of its endpoints is in S ?
- Ex. Is there an independent set of size ≥ 6 ? Yes.
- Ex. Is there an independent set of size ≥ 7 ? No.



● independent set

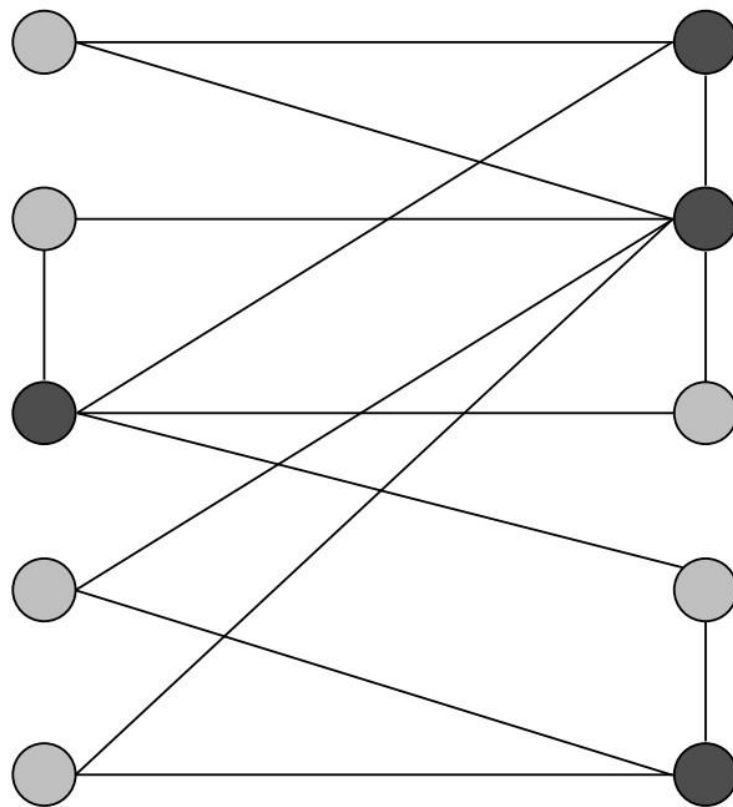




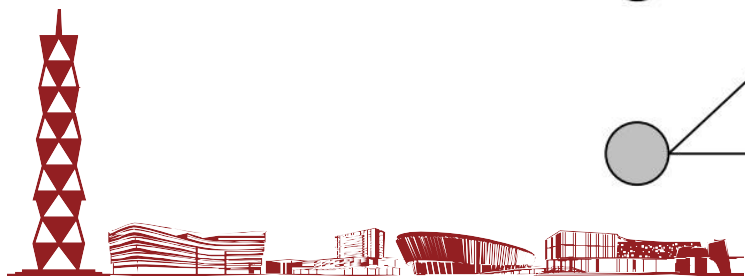
Vertex Cover



- **Vertex Cover.** Given a graph $G = (V, E)$ and an integer k , is there a subset of vertices $S \subseteq V$ such that $|S| \leq k$, and for each edge, at least one of its endpoints is in S ?
- Ex. Is there a vertex cover of size ≤ 4 ? Yes.
- Ex. Is there a vertex cover of size ≤ 3 ? No.



● vertex cover

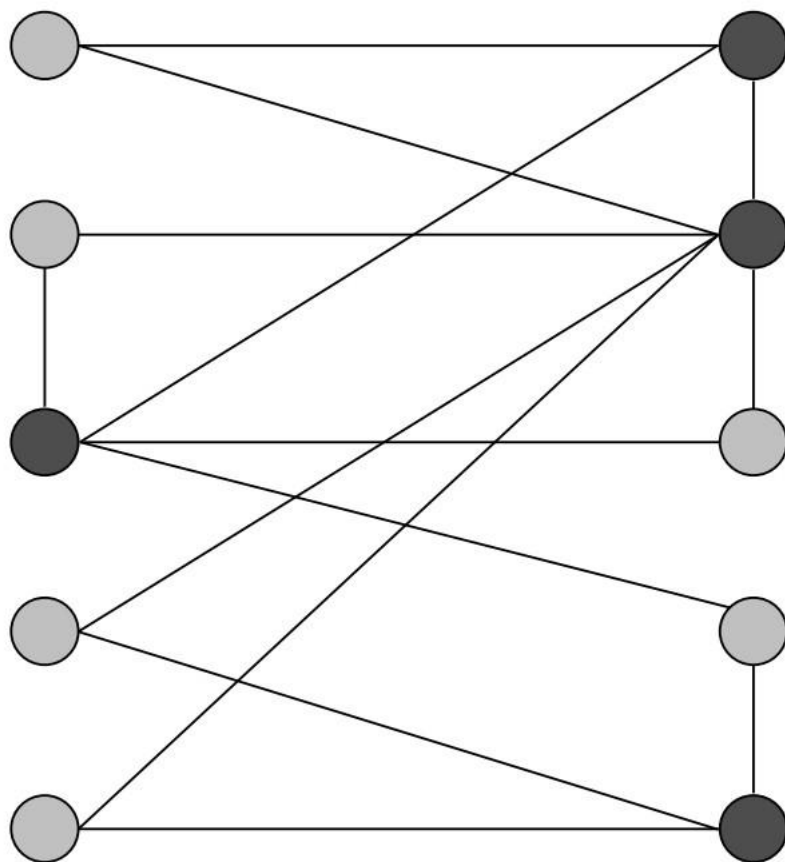




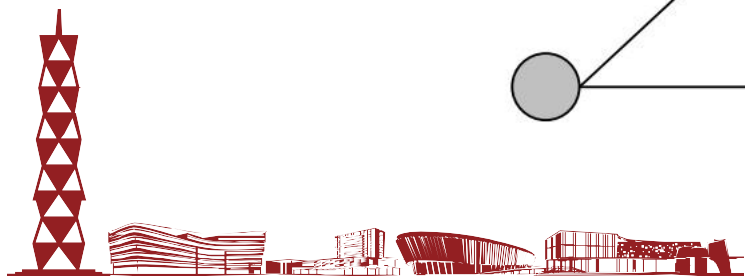
Vertex Cover and Independent Set



- **Claim.** Vertex-cover $\equiv_p$ Independent-set
- **Pf.** We show S is an independent set iff $V - S$ is a vertex cover



- independent set
- vertex cover





Vertex Cover and Independent Set



- **Claim.** Vertex-cover $\equiv_p$ Independent-set
- **Pf.** We show S is an independent set iff $V - S$ is a vertex cover
- $\rightarrow$
 - Let S be any independent set
 - Consider an arbitrary edge (u, v)
 - S independent $\rightarrow u \notin S$ or $v \notin S \rightarrow u \in V - S$ or $v \in V - S$
 - Thus, $V - S$ covers (u, v)
- $\leftarrow$
 - Let $V - S$ be any vertex cover
 - Consider two nodes $u \in S$ and $v \in S$
 - Observe that $(u, v) \notin E$ since $V - S$ is a vertex cover
 - Thus, no two nodes in S are joined by an edge $\rightarrow S$ independent set

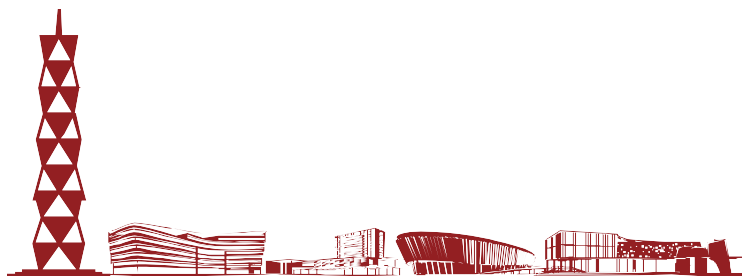




Reduction by Simple Equivalence



- **Basic reduction strategies**
 - Reduction by simple equivalence
 - **Reduction from special case to general case**
 - Reduction by encoding with gadgets





Set Cover



- **Set Cover.** Given a set U of elements, a collection $S_1, S_2, \dots, S_m$ of subsets of U , and an integer k , does there exist a collection of $\leq k$ of these sets whose union is equal to U ?
- **Sample application.**
 - m available pieces of software
 - Set U of n capabilities that we would like our system to have
 - The i -th piece of software provides the set $S_i \subseteq U$ of capabilities
 - Goal: achieve all n capabilities using fewest pieces of software

$$U = \{ 1, 2, 3, 4, 5, 6, 7 \}$$

$$k = 2$$

$$S_1 = \{ 3, 7 \}$$

$$S_4 = \{ 2, 4 \}$$

$$S_2 = \{ 3, 4, 5, 6 \}$$

$$S_5 = \{ 5 \}$$

$$S_3 = \{ 1 \}$$

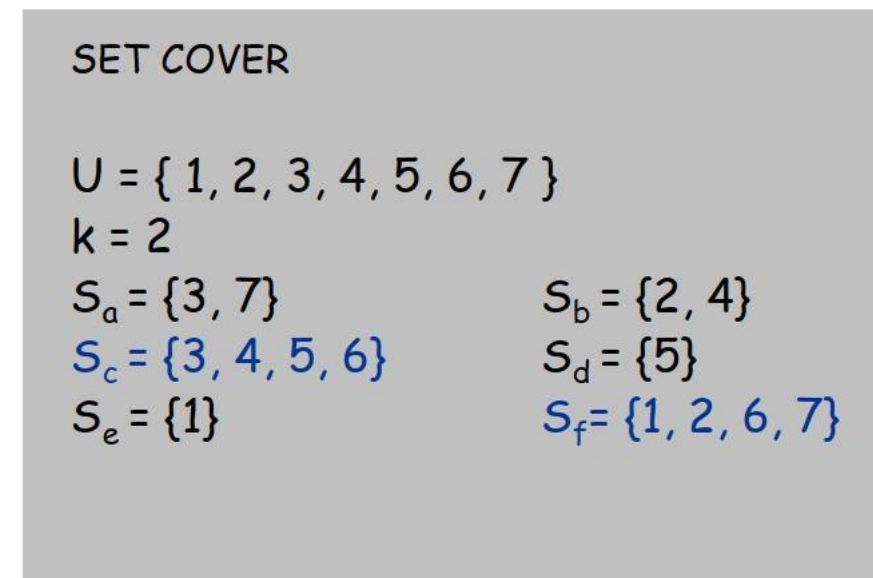
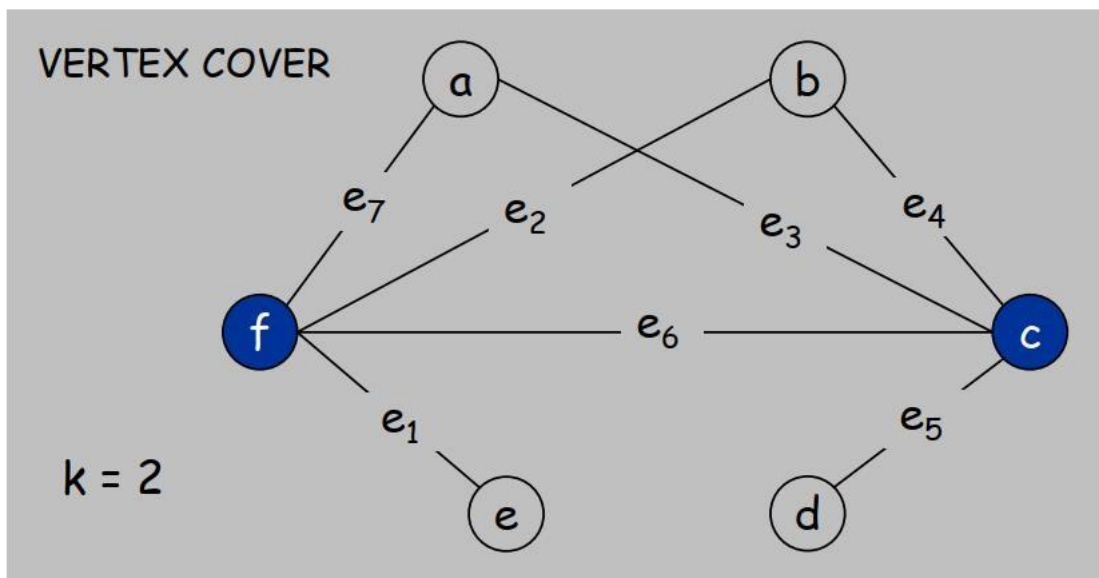
$$S_6 = \{ 1, 2, 6, 7 \}$$





Vertex Cover Reduces to Set Cover

- **Claim.** Vertex-cover $\leq_p$ Set-cover
- **Pf.** Given a vertex-cover instance $G = (V, E)$, k , we construct a set cover instance (U, S) whose size equals to the size of the vertex cover instance
- **Construction.**
 - Create Set-cover instance:
 - $k = k$, $U = E$, $S_v = \{e \in E: e \text{ incident to } v\}$
 - Set-cover of size $\leq k$ iff vertex cover of size $\leq k$

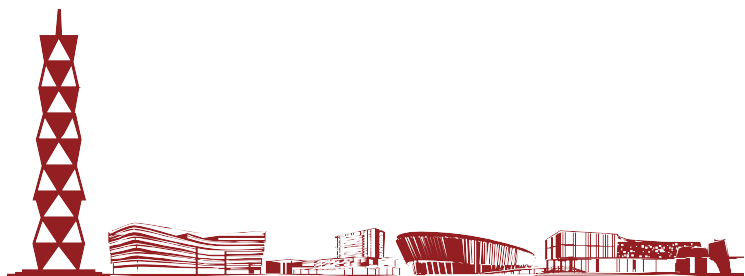




Reduction by Simple Equivalence



- **Basic reduction strategies**
 - Reduction by simple equivalence
 - Reduction from special case to general case
 - **Reduction by encoding with gadgets**





Satisfiability



- **Literal:** A Boolean variable or its negation

$$x_i \text{ or } \overline{x_i}$$

- **Clause:** A disjunction of literals

$$C_j = x_1 \vee \overline{x_2} \vee x_3$$

- **Conjunctive normal form:** A propositional formula Φ that is the conjunction of clauses

$$\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$$

- **SAT:** Given CNF formula Φ , does it have a satisfying truth assignment?
- **3-SAT:** SAT where each clause contains exactly 3 literals

← each corresponds to a different variable

Ex: $(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$

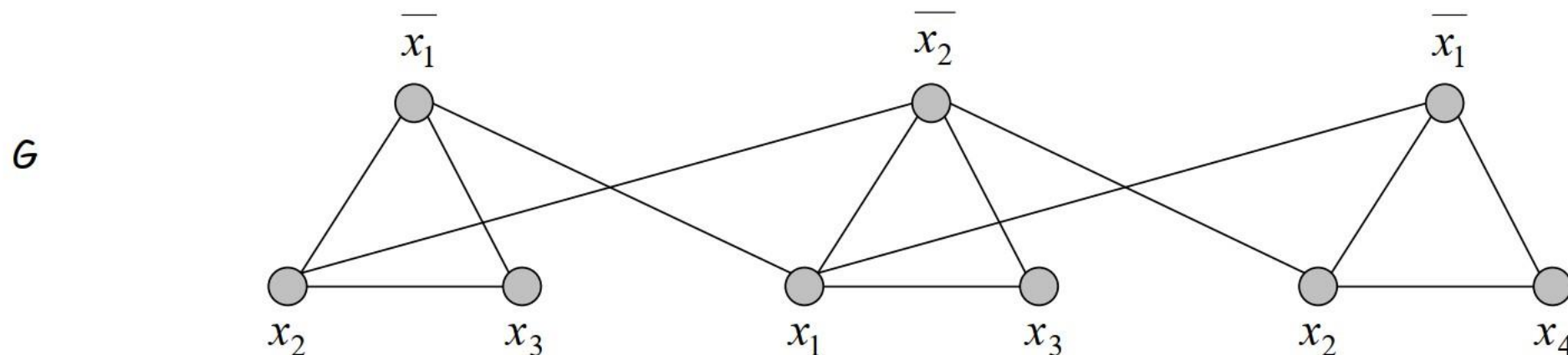
Yes: $x_1 = \text{true}, x_2 = \text{true}, x_3 = \text{false}, x_4 = \text{false}.$





3 Satisfiability Reduces to Independent Set

- **Claim.** 3-SAT $\leq_p$ Independent Set
- **Pf.** Given an instance Φ of 3-SAT, we construct an instance (G, k) of independent-set that has an independent set of size k iff Φ is satisfiable
- **Construction**
 - G contain 3 vertices for each clause, one for each literal
 - Connect 3 literals in a clause in a triangle
 - Connect literal to each of its negations



$k = 3$

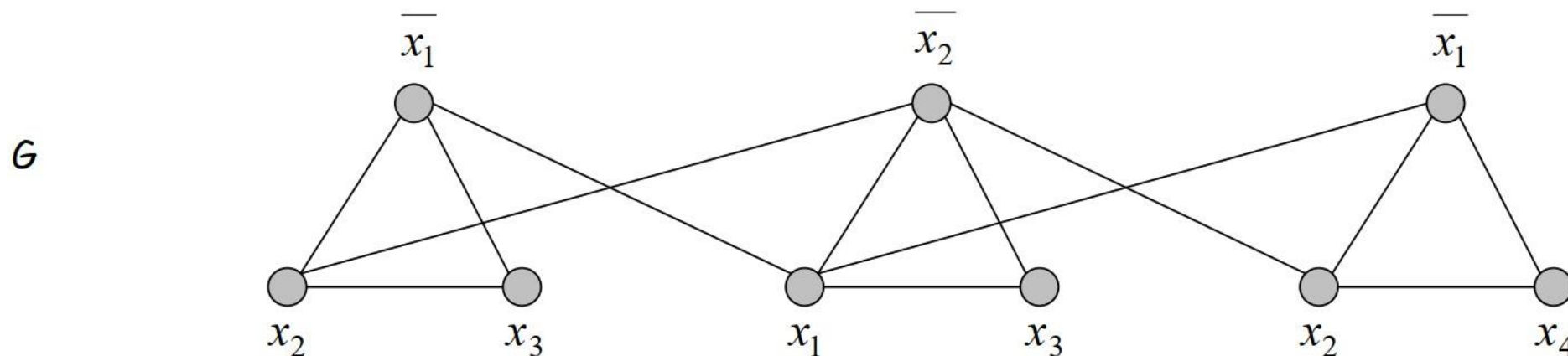
$$\Phi = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_4)$$





3 Satisfiability Reduces to Independent Set

- **Claim.** G contains independent set of size $k = |\Phi|$ iff Φ is satisfiable
- **Pf. $\rightarrow$** Let S be independent set of size k
 - S must contain exactly one vertex in each triangle
 - Set these literals to true $\leftarrow$ and any other variables in a consistent way
 - Truth assignment is consistent and all clauses are satisfied
- **Pf. $\leftarrow$** Given satisfying assignment, select one true literal from each triangle. This is an independent set of size k



$k = 3$

$$\Phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$$





Review



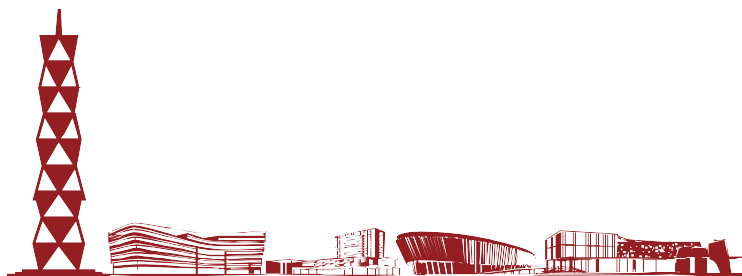
- **Basic reduction strategies**

- Simple equivalence: independent-set $\equiv_p$ Vertex-cover
- Special case to general case: Vertex-cover $\leq_p$ set-cover
- Encoding with gadgets: 3-SAT $\leq_p$ Independent-set

- **Transitivity.** If $x \leq_p y$ and $y \leq_p z$, then $x \leq_p z$

- **Pf idea.** Compose the two algorithms

- **Ex:** 3-SAT $\leq_p$ Independent-set $\leq_p$ Vertex-cover $\leq_p$ Set-cover



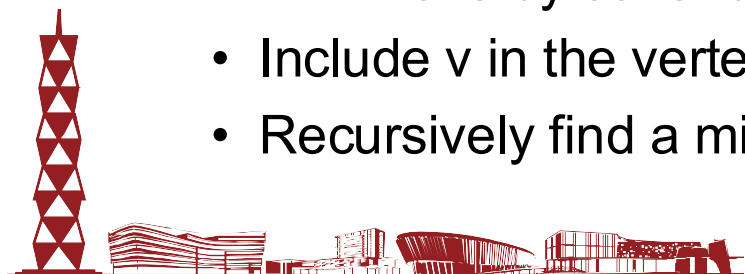


Self-Reducibility



- **Decision problem.** Does there **exist** a vertex cover of size $\leq k$?
- **Optimization problem.** **Find** vertex cover of minimum cardinality
- **Self-reducibility.** Optimization problem $\leq$ decision version
 - Applies to all (NP-complete) problems here
 - Justifies our focus on decision problems
- **Ex: to find min cardinality vertex cover**
 - (Binary) search for cardinality k^* of min vertex cover
 - Find a vertex v such that $G - \{v\}$ has a vertex cover of size $\leq k^* - 1$
 - v must be in a vertex cover of size $\leq k^*$
 - Prove by construction: a vertex cover of $G - \{v\}$ plus v must be a vertex cover of G
 - Include v in the vertex cover
 - Recursively find a min vertex cover in $G - \{v\}$

delete v and all incident edges





Find min cardinality vertex cover



- Given a graph G and an integer k , decide whether G has a vertex cover of size at most k .
- **Step 1:** Find the minimum size k^*
 - Querying the decision algorithm on different values of k .
 - Since the answer changes from **NO** to **YES** at the optimum value, we can use binary search over $k \in [0, |V|]$ to find

$k^* = \text{the size of a minimum vertex cover of } G.$

Step 2: Identify one vertex that must belong to some optimal solution

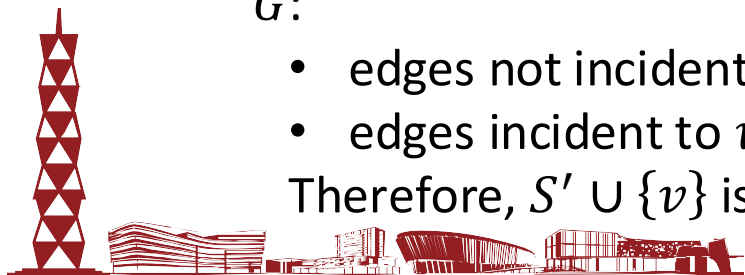
- Now we try to find a vertex v such that

$$G - \{v\}$$

has a vertex cover of size at most $k^* - 1$.

- If $G - \{v\}$ has a vertex cover of size at most $k^* - 1$, then adding v back gives a vertex cover of G of size at most k^* .
- Let S' be a vertex cover of $G - \{v\}$ with $|S'| \leq k^* - 1$. Then $S' \cup \{v\}$ covers every edge of G :
 - edges not incident to v are already covered by S' ,
 - edges incident to v are covered by v .

Therefore, $S' \cup \{v\}$ is a vertex cover of G of size at most k^* .

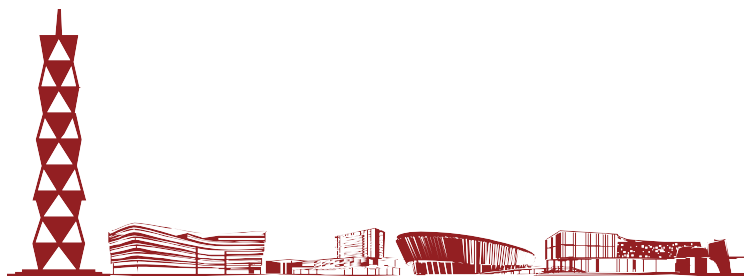




Find min cardinality vertex cover

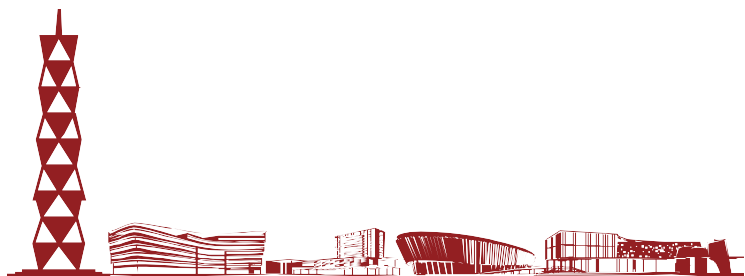


- Given a graph G and an integer k , decide whether G has a vertex cover of size at most k .
- **Step 3:** Add v to the solution $C=\{v\}$.
- **Step 4: Recurse on the smaller graph**
 - Now solve the same problem recursively on $G - \{v\}$, with target size $k^* - 1$.
 - That is, we now need to find a minimum vertex cover of size $k^* - 1$ in the reduced graph.
 - Repeat the same logic





Definition of P vs.NP



- **Decision problem**

- X is a set of strings
- Instance: string s
- Algorithm A solves problem X : $A(s) = \text{yes}$ iff $s \in X$

- **Polynomial time.** Algorithm A runs in polynomial time if for every string s , $A(s)$ terminates in at most $p(|s|)$ “step”, where $p(\cdot)$ is some polynomial

↑
length of s

- **PRIMES:** $X = \{2, 3, 5, 7, 11, 13, 17, 23, 29, 31, 37, \dots\}$
- **Instance:** $s = 592335744548702854681$
- **Algorithm:** [Agrawal-Kayal-Saxena, 2002] $p(|s|) = |s|^8$



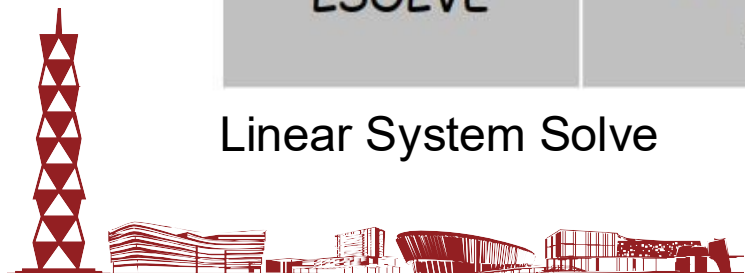
Definition of P



- P. Decision problems for which there is a poly-time algorithm

Problem	Description	Algorithm	Yes	No
MULTIPLE	Is x a multiple of y ?	Grade school division	51, 17	51, 16
RELPRIME	Are x and y relatively prime?	Euclid (300 BCE)	34, 39	34, 51
PRIMES	Is x prime?	AKS (2002)	53	51
EDIT-DISTANCE	Is the edit distance between x and y less than 5?	Dynamic programming	niether neither	acgggt ttttta
LSOLVE	Is there a vector x that satisfies $Ax = b$?	Gauss-Edmonds elimination	$\begin{bmatrix} 0 & 1 & 1 \\ 2 & 4 & -2 \\ 0 & 3 & 15 \end{bmatrix}, \begin{bmatrix} 4 \\ 2 \\ 36 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$

Linear System Solve





- **Certification algorithm intuition**

- Certifier views things from “managerial” viewpoint
- Certifier doesn’t determine whether $s \in X$ on its own; rather, it checks a proposed proof t that $s \in X$

- **Def.** Algorithm $C(s, t)$ is a **certifier** for problem X if for every string s , $s \in X$ iff there exists a string t such that $C(s, t) = \text{yes}$



“certificate” or “witness”

- **NP.** Decision problems for which there exists a **poly-time** certifier
 - $C(s, t)$ is a poly-time algorithm
 - $|t| \leq p(|s|)$ for some polynomial $p(\cdot)$

- **Remark.** NP stands for **nondeterministic** polynomial-time





Certifiers and Certificates: Composite



- **COMPOSITES.** Given an integer s , is s composite?
- **Certificate.** A nontrivial factor t of s . Note that such a certificate exists iff s is composite. Moreover $|t| \leq |s|$ ($s \in X \iff \exists t$ such that $C(s, t) = \text{yes}$)
- **Certifier.**

```
boolean C(s, t) {  
    if (t ≤ 1 or t ≥ s)  
        return false  
    else if (s is a multiple of t)  
        return true  
    else  
        return false  
}
```

- **Instance.** $s = 437,669$
- **Certificate.** $t = 541$ or 809 $\longleftarrow 437,669 = 541 * 809$

Certifier check:

$$541 > 1$$

$$541 < 437,669$$

$$437,669 \bmod 541 = 0$$

- **Conclusion:** COMPOSITES is in NP





Certifiers and Certificates: 3-Satisfiability



- **SAT.** Given a CNF formula ϕ , is there a satisfying assignment?
- **Certificate.** An assignment of truth values to the n Boolean variables
- **Certifier.** Check that each clause in ϕ has at least one true literal
- **Ex.**

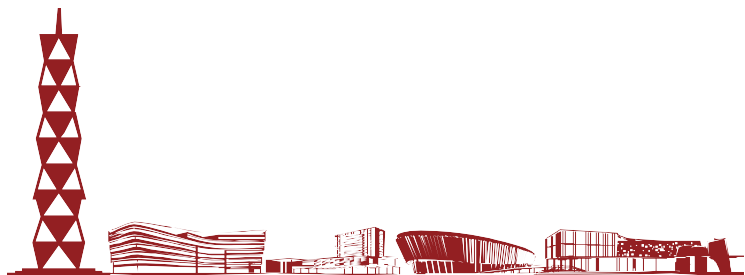
$$\left(\overline{x_1} \vee x_2 \vee x_3\right) \wedge \left(x_1 \vee \overline{x_2} \vee x_3\right) \wedge \left(x_1 \vee x_2 \vee x_4\right) \wedge \left(\overline{x_1} \vee \overline{x_3} \vee \overline{x_4}\right)$$

instance s

$$x_1 = 1, x_2 = 1, x_3 = 0, x_4 = 1$$

certificate t

- **Conclusion.** SAT is in NP

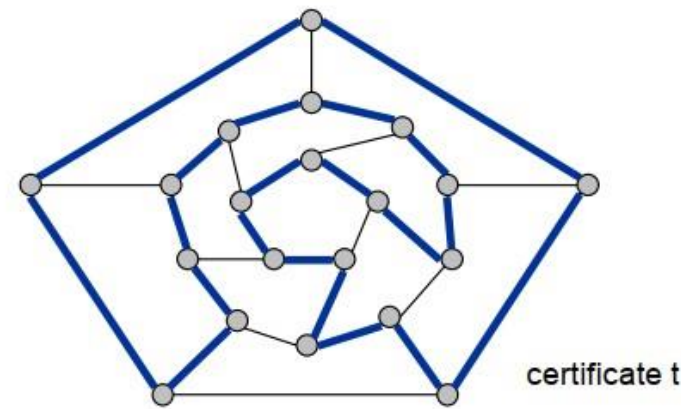
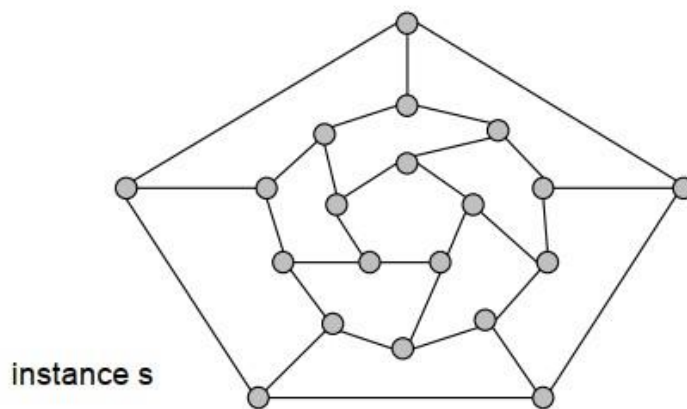




Certifiers and Certificates: Hamiltonian Cycle



- **HAM-CYCLE** Given an undirected graph $G = (V, E)$, does there exist a simple cycle C that visits every node?
- **Certificate**. A permutation of the n nodes
- **Certifier**. Check that the permutation contains each node in V exactly once, and that there is an edge between each pair of adjacent nodes in the permutation
- **Conclusion**. HAM-CYCLE is in NP





Definition of NP



- NP.** Decision problems for which there exists a poly-time certifier.

Problem	Description	Algorithm	Yes	No
LSOLVE	Is there a vector x that satisfies $Ax = b$?	Gauss-Edmonds elimination	$\left[\begin{array}{ccc c} 0 & 1 & 1 & 4 \\ 2 & 4 & -2 & 2 \\ 0 & 3 & 15 & 36 \end{array} \right]$	$\left[\begin{array}{ccc c} 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{array} \right]$
PRIMES	Is x prime?	AKS (2002)	53	51
FACTOR	Does x have a nontrivial factor less than y ?	?	(56159, 50)	(55687, 50)
SAT	Is there a truth assignment that satisfies the formula?	?	$\neg x_1 \vee x_2$ $x_1 \vee x_2$	$\neg x_2$ $\neg x_1 \vee x_2$ $x_1 \vee x_2$
HAM-CYCLE	Does there exist a simple cycle C that visits every node?	?		





P, NP, EXP



- **P.** Decision problems for which there is a **poly-time algorithm**
- **EXP.** Decision problems for which there is an **exponential-time algorithm**
- **NP.** Decision problems for which there is a **poly-time certifier**

- **Claim.** $P \subseteq NP$
- **Pf.** Consider any problem X in P
 - By definition, there exists a poly-time algorithm $A(s)$ that solves X
 - Certificate: $t = \varepsilon$, certifier $C(s, t) = A(s)$
- **Claim.** $NP \subseteq EXP$
- **Pf.** Consider any problem X in NP
 - By definition, there exists a poly-time certifier $C(s, t)$ for X
 - To solve input s , run $C(s, t)$ on all strings t with $|t| \leq p(|s|)$
 - Return yes, if $C(s, t)$ returns yes for any of these

Because the number of possible certificates
of length at most $p(n)$ is roughly:
 $2^{p(n)}$
which is exponential.

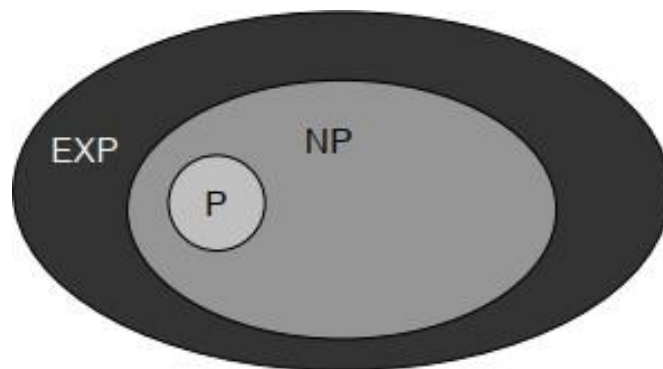




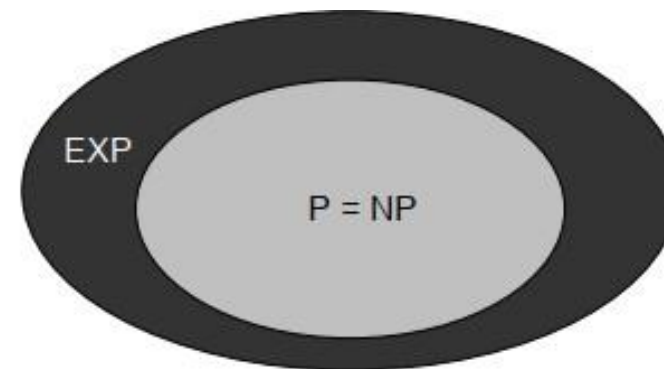
The Main Question: P Versus NP



- **Does $P = NP$?** [Cook 1971, Edmonds, Levin, Yablonski, Godel]
 - Is the decision problem as easy as the certification?



If $P \neq NP$



If $P = NP$

Would break RSA cryptography (and potentially collapse economy)



- **If yes:** Efficient algorithms for 3-COLOR, TSP, SAT, FACTOR, ...
- **If no:** No efficient algorithms possible for 3-COLOR, TSP, SAT, ...





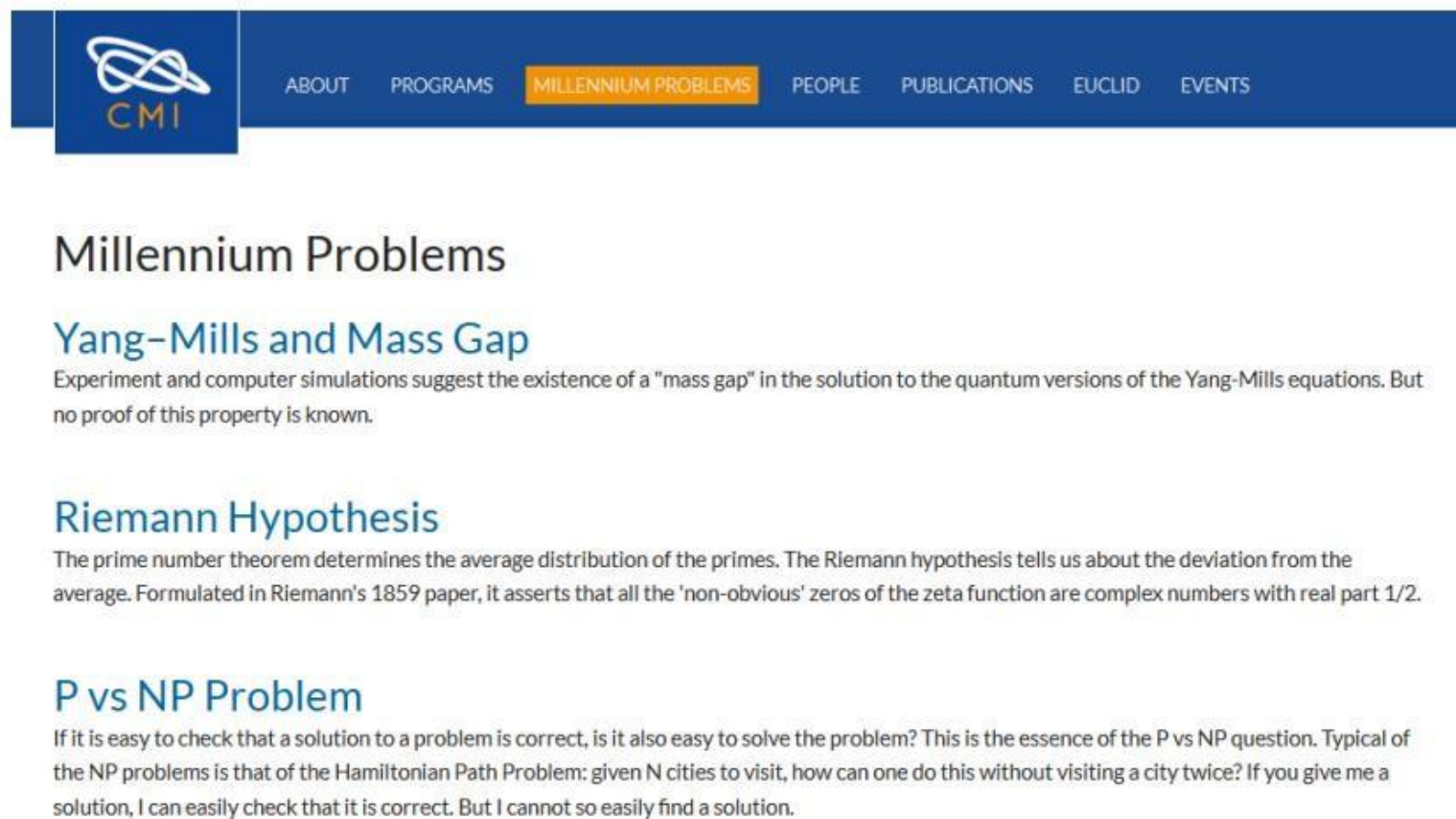
- **Consensus opinion on $P = NP$?** Probably no
- 2002 poll of 100 researchers
 - 61: no
 - 9: yes
 - 22: unsure
 - 4: the question may be independent of the currently accepted axioms and therefore is impossible to prove or disprove
- 2012 poll of 151 researchers
 - 126: no
 - 12: yes
 - 8: don't know / don't care
 - 5: the question may be independent of the currently accepted axioms and therefore is impossible to prove or disprove



Millennium prize

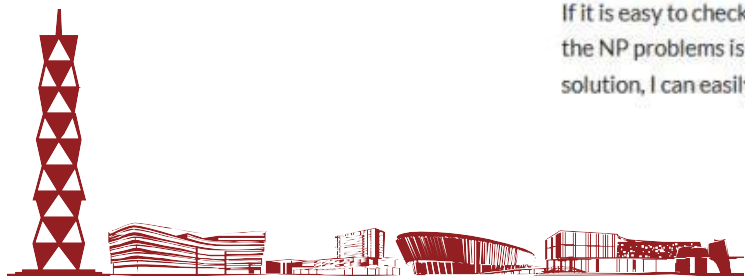


- Millennium prize. \$1 million for resolution of $P \neq NP$ problem.



The screenshot shows the CMI Millennium Problems website. At the top is a blue navigation bar with the CMI logo (a white knot) and the text "CMI" on the left, and a series of menu items: "ABOUT", "PROGRAMS", "MILLENNIUM PROBLEMS" (highlighted in orange), "PEOPLE", "PUBLICATIONS", "EUCLID", and "EVENTS". Below the navigation bar is the heading "Millennium Problems". Under this heading are three sections, each with a title and a brief description:

- Yang-Mills and Mass Gap**
Experiment and computer simulations suggest the existence of a "mass gap" in the solution to the quantum versions of the Yang-Mills equations. But no proof of this property is known.
- Riemann Hypothesis**
The prime number theorem determines the average distribution of the primes. The Riemann hypothesis tells us about the deviation from the average. Formulated in Riemann's 1859 paper, it asserts that all the 'non-obvious' zeros of the zeta function are complex numbers with real part $1/2$.
- P vs NP Problem**
If it is easy to check that a solution to a problem is correct, is it also easy to solve the problem? This is the essence of the P vs NP question. Typical of the NP problems is that of the Hamiltonian Path Problem: given N cities to visit, how can one do this without visiting a city twice? If you give me a solution, I can easily check that it is correct. But I cannot so easily find a solution.

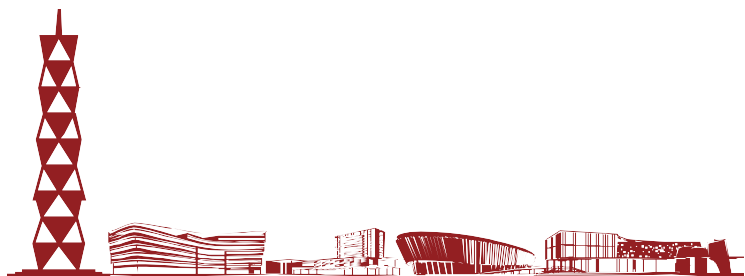




Princeton CS Building, West Wall



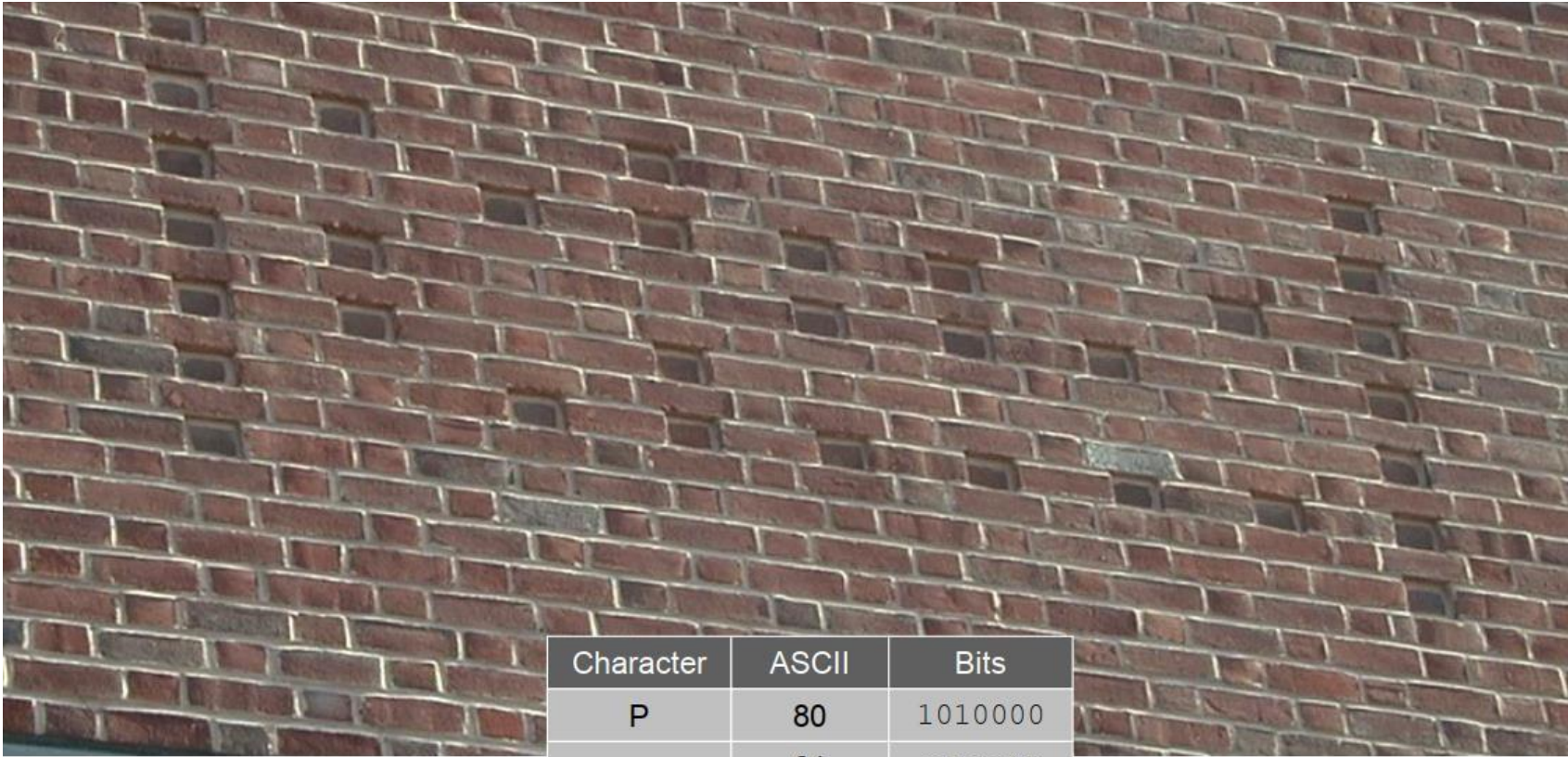
上海科技大学
ShanghaiTech University



立志成才 报 国 裕 民



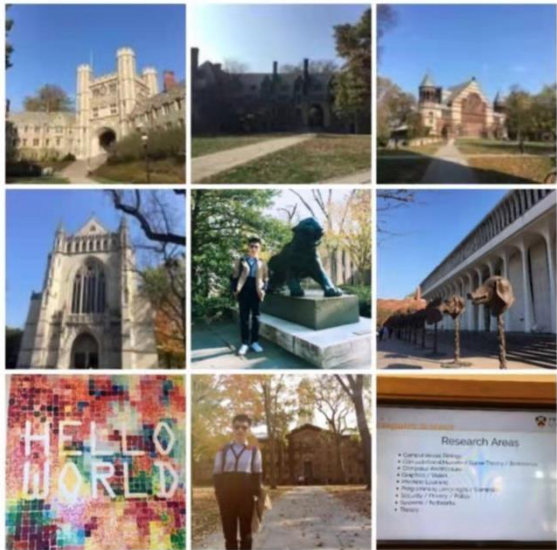
Princeton CS Building, West Wall



Character	ASCII	Bits
P	80	1010000
=	61	0111101
N	78	1001110
P	80	1010000
?	63	0111111



「普林斯顿大学」人的一生如果能有几年来这里静心做几年自己喜爱的事业，该是多么幸福的事情！



美国 · Princeton Township · Computer Science Building, Prince...

2016年10月31日 10:37 删除



2016年10月31日 11:34

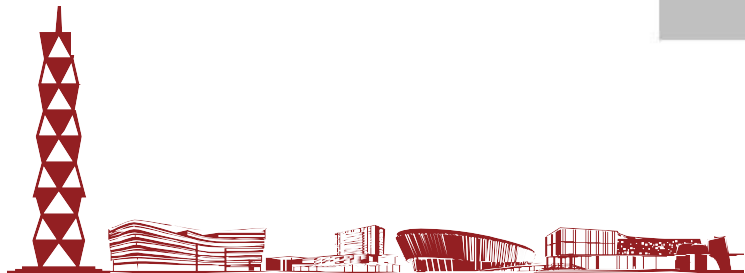
有没有看到计算机系外墙上的密码？



2016年10月31日 11:38

凸起的砖块表示用二进制描述的P和

NP问题😄





Next Time: NP and Computational Intractability (Cont.)

